Trabalho 2 AEDS I

Teste de complexidade para vetor e lista para as entidades Aluno e Professor

Aluno.C

Para a entidade aluno é criada as funções principais pedidas, isto é:

- Inserir
- Remover
- Buscar
- Listar

Além de funções auxiliares e de iniciação como:

- Iniciar vetor aluno
- Iniciar entidade aluno
- Imprimir linha de aluno

Cada uma possui a seguinte complexidade que se segue pelas funções a seguir:

1. Iniciar / Inserir

Tal função apenas recebe valores e os aloca em seus respectivos espaços em uma determinada entidade criada dentro do vetor, ocupando um espaço pré delimitado.

A função “memset” , por exemplo, preenche um espaço S de bytes, como esse espaço S é uma constante pode se dizer que, **a complexidade é constante, isto é, O(1).**

```
memset(a, 0, sizeof(*a)
```

Para as seguintes funções de alocação de variáveis se dá pela capacidade do campo, no caso do exemplo a seguir, o campo nome, como esse campo já está definida como constante no arquivo “Util.h” a complexidade também é constante e isso se segue para todas as outras, sendo assim **O(1).**

```
a->matricula = matricula;
strncpy(a->nome, nome, sizeof(a->nome) - 1);
```

Já a de inserir apenas coloca está entidade criada no vetor possuindo igual complexidade O(1), além de que a função de iniciar vetor, que não falei sobre, **possui mesma complexidade O(1).**

2. Buscar

Tal função percorre o vetor de alunos para encontrar um determinado aluno pelo código de matrícula passado.

Para o seu melhor caso, isto é, se o aluno matriculado estiver na primeira posição do vetor, assim o for só precisa ser rodado uma única vez, assumindo uma complexidade constante: **O(1).**

```
    for(int i = 0; i < v->tamanho; i++){
        if(v->dados[i].matricula == matricula){
            return i;
        }
    }
    return -1;
```

Para o seu pior caso seria se não tivesse aluno no vetor, precisando rodar o “for” pela quantidade de vezes que o vetor possui de tamanho. Assim faria uma inicialização de i, que é constante, faz N+1 verificações da condição “for”, N comparações de matrículas, faz N incrementos a variável i, não haveria return i visto que não encontrou o aluno, logo termina retornando -1, assim a função de complexidade ficaria:

F(N) = c1 + (N+1)c2 + Nc3 + Nc4 + c5
F(N) = N(c2+c3+c4) + (c1+c2+c5)
F(N) = Na + b

Ignorando as constantes na análise, temos que **a complexidade no pior caso é: O(N).**

Para o caso médio, supondo que o aluno exista e que todas as posições sejam igualmente prováveis, temos a soma de cada uma das posições possíveis dividido pela quantidade de possibilidades, isto é, N:

F(N) = (1 + 2 + 3 + …. + N) / N
F(N) = ((N(N+1)) / 2) / N
F(N) = (N+1) / 2
F(N) = a(N+1) / 2 + b

Ignorando as constantes na análise, temos que a complexidade no caso médio é: O(N). Assim sendo **a complexidade geral do função é dada pela taxa de Crescimento O(N)**

3. Remover

A função busca o aluno e depois o removê através de um for que percorre do ponto em que o aluno está até o final fazendo com que se duplique o próximo para o anterior, e depois diminui o tamanho do vetor:

```
    int i = buscarA(v, matricula);

    if(i != -1){

    for(int j = i; j < v->tamanho - 1; j++){
        v->dados[j] = v->dados[j + 1];
    }
    v->tamanho--;
    return 1;
    }
    return 0;
```

Se o aluno está na posição i (começando em 0), a busca faz i+1 comparações e o deslocamento faz N−1−i cópias. O custo total é:

(i+1) + (N−1−i) = N

Ou seja, o total é N operações, qualquer que seja a posição i. Quanto mais cedo o aluno é encontrado, mais elementos precisam ser deslocados, e vice-versa. As duas partes se compensam.

Melhor caso. Não existe melhor caso constante: com o aluno na primeira posição, a busca é barata (1 comparação), mas o deslocamento copia N−1 elementos; com o aluno na última posição, a busca percorre N elementos e o deslocamento não copia nada. Em ambos o custo é da ordem de N. **O melhor caso é, portanto, O(N).**

Pior caso. O custo é N operações tanto quando o aluno existe (em qualquer posição) quanto quando não existe (N comparações e nenhum deslocamento):

F(N) = c1 + Nc2 + c3

F(N) = aN + b

O pior caso é O(N).

Caso médio. Com as posições igualmente prováveis, a busca faz em média (N+1)/2 comparações e o deslocamento faz em média (N−1)/2 cópias:

F(N) = a(N+1)/2 + b(N−1)/2 + c

Para a = b, F(N) = aN + c, que é O(N).

Como os três casos têm a mesma ordem de crescimento, a função é O(N): **a complexidade geral da função é de crescimento O(N).**

4. Listar

A função percorre todo o vetor e, para cada aluno, chama a função de imprimir. A impressão de uma linha de aluno tem custo constante, O(1), pois depende apenas do tamanho fixo dos campos. Como ela é chamada uma vez para cada um dos N alunos:

```
for(int i = 0; i < v->tamanho; i++){
    imprimirAluno(&v->dados[i]);
}
```

F(N) = c1 + N(c2 + c3) + c4

F(N) = aN + b

A função sempre percorre todos os N elementos, independentemente dos dados. Não há melhor nem pior caso distintos: **o custo é O(N) nos três casos**. (Um vetor com um único aluno não é um melhor caso, é apenas um N menor, pois a complexidade é uma função de N.)

5. Complexidade geral de Aluno.C

Cada operação é analisada separadamente, pois elas não são executadas em conjunto. A operação mais cara define a ordem de crescimento do módulo:

- Iniciar vetor / aluno: melhor caso O(1), caso médio O(1), pior caso O(1).
- Inserir: melhor caso O(1), caso médio O(1), pior caso O(1).
- Buscar: melhor caso O(1), caso médio O(N), pior caso O(N).
- Remover: melhor caso O(N), caso médio O(N), pior caso O(N).
- Listar: melhor caso O(N), caso médio O(N), pior caso O(N).

As operações de iniciar e inserir têm custo constante, enquanto buscar, remover e listar crescem linearmente com N. Logo, Aluno.c implementado com vetor **possui complexidade O(N) para suas operações mais custosas, e O(1) para iniciar e inserir.**

Professor.c

Para a entidade professor, implementada com lista duplamente encadeada, são criadas as funções principais pedidas, isto é:

- Inserir
- Remover
- Buscar
- Listar

Além de funções auxiliares e de iniciação como:

- Iniciar lista de professores
- Iniciar entidade professor
- Imprimir linha de professor
- Liberar lista de professores

Cada uma das funções possui a seguinte complexidade, que se segue pelas funções a seguir:

1. Iniciar/Inserir

A função de iniciar a lista apenas coloca os ponteiros `inicio` e `fim` em `NULL` e zera o tamanho, ou seja, três atribuições de custo constante, **O(1):**

```
    l->inicio = l->fim = NULL;
    l->tamanho = 0;
```

A função de iniciar o professor recebe valores e os copia para os respectivos campos da entidade. Sendo assim possui complexidade **O(1):**

```
    p->matricula = matricula;

    strncpy(p->nome,  nome,  TAM_NOME  - 1);

    p->nome[TAM_NOME - 1]   = '\0';
```

Já a função de inserir cria um novo nó com `malloc` (considerado de custo constante), copia o professor para dentro dele e o liga ao final da lista. Como a lista guarda o ponteiro `fim`, não é preciso percorrer nenhum nó para chegar ao último, e a inserção tem custo **O(1)** em todos os casos.

Diferente do vetor, a lista não tem capacidade máxima fixa, então não existe o teste de espaço disponível. Só há dois caminhos possíveis (lista vazia ou não), ambos com o mesmo custo constante.

2. Buscar

Tal função percorre a lista, nó a nó, para encontrar um determinado professor pelo código de matrícula passado. Ela devolve o endereço do nó encontrado ou NULL, no lugar do índice que o vetor devolvia, pois a lista não possui posições numeradas:

```
for(NoProfessor *at = l->inicio; at; at = at ->prox){

    if(at->dados.matricula == matricula){

        return at;

    }

}

return NULL;
```

Constantes usadas: c1 = inicialização de `at`; c2 = teste da condição do `for`; c3 = avanço para o próximo nó; c4 = comparação de matrícula; c5 = `return`.

Melhor caso. O professor está no primeiro nó. O laço executa uma única vez e retorna, com custo constante: **O(1)**

Pior caso**.** O professor não existe na lista (ou está no último nó). O laço faz N+1 testes da condição, N comparações de matrícula e N avanços, e termina em `return NULL`:

F(N) = c1 + (N+1)c2 + Nc3 + Nc4 + c5

F(N) = N(c2+c3+c4) + (c1+c2+c5)

F(N) = aN + b

Ignorando as constantes, **o pior caso é O(N).**

Caso médio. Supondo que o professor exista e que todas as posições sejam igualmente prováveis, o número médio de comparações é:

F(N) = (1 + 2 + 3 + ... + N) / N

F(N) = (N(N+1)/2) / N

F(N) = (N+1)/2

Como cada iteração custa uma constante a, o custo médio é a(N+1)/2 + b, que é **O(N)**.

Portanto, a complexidade da função é de crescimento **O(N)**. A ordem é a mesma da busca no vetor. A diferença é que, no vetor, o acesso a `dados[i]` é direto, enquanto na lista é preciso seguir os ponteiros um a um para chegar ao próximo elemento.

3. Remover

A função busca o professor e, se ele existir, religa os nós vizinhos de modo que ninguém mais aponte para ele. Depois libera a memória do nó e diminui o tamanho da lista. Se a matrícula não existir, a função retorna 0 sem alterar a lista:

```
NoProfessor *at = buscarP(l, matricula);

if(at->ant){
    at->ant->prox = at->prox;
}else{
    l->inicio = at->prox;
}

if(at->prox){
    at->prox->ant = at->ant;
}else{
    l->fim = at->ant;
}

free(at);
l->tamanho--;
return 1;
```

Diferente do vetor, **não há deslocamento de elementos**. Como o nó já tem o ponteiro `ant`, religar os vizinhos exige um número fixo de atribuições, independente de N. O custo total é então o custo da busca B(N) somado a um custo constante c (dois `if/else`, o `free`, o decremento e o `return`):

F(N) = B(N) + c

Melhor caso**.** O professor está no primeiro nó: a busca faz uma única comparação e a remoção apenas religa `inicio`. O custo é constante: **O(1)**.

Pior caso. O professor está no último nó ou não existe. A busca faz N comparações e o restante é constante (quando não existe, a função retorna logo após a busca):

F(N) = Nc1 + c2

F(N) = aN + b

O pior caso é **O(N)**.

Caso médio. Com as posições igualmente prováveis, a busca faz em média (N+1)/2 comparações e o restante é constante:

F(N) = a(N+1)/2 + b

O caso médio é **O(N)**.

A complexidade geral da função é de crescimento **O(N)**, dominada pela busca. Note a diferença em relação ao vetor: lá o custo era **O(N)** em todos os casos, pois (i+1) comparações somadas a (N−1−i) cópias dão sempre N. Na lista, a remoção tem melhor caso constante, pois a parte de deslocar elementos não existe. **Se já se tivesse o ponteiro do nó, a remoção seria O(1),** já que o custo N vem apenas da função de busca, enquanto a remoção é constante.

4. Listar

A função percorre todos os nós da lista e, para cada professor, chama a função de imprimir. **A impressão de uma linha de professor tem custo constante, O(1)**, pois depende apenas do tamanho fixo dos campos. Como ela é chamada uma vez para cada um dos N professores:

```
    for(NoProfessor *at = l->inicio; at; at = at->prox){
        imprimirProfessor(&at->dados);
    }
```

F(N) = c1 + N(c2 + c3) + c4

F(N) = aN + b

A função sempre percorre todos os N nós, independentemente dos dados. Não há melhor nem pior caso distintos: o custo é **O(N)** nos três casos. (Uma lista com um único professor não é um melhor caso, é apenas um N menor, pois a complexidade é uma função de N.)

5. Liberar

Esta função não existe no vetor. Como cada nó foi criado com `malloc`, é preciso liberar todos eles no fim do programa. A função percorre a lista guardando o próximo nó antes de liberar o atual, para não perder o encadeamento:

```
    NoProfessor *at = l->inicio;
    while (at) {
        NoProfessor *prox = at->prox;
        free(at);
        at = prox;
    }

    l->inicio = l->fim = NULL;
    l->tamanho = 0;
```

O laço executa exatamente N vezes, cada uma com custo constante (guardar o próximo, liberar o nó e avançar):

F(N) = c1 + N(c2 + c3 + c4) + c5

F(N) = aN + b

O custo é **O(N)** nos três casos, pois sempre percorre todos os nós.

6. Complexidade geral de Professor.c

Cada operação é analisada separadamente, pois elas não são executadas em conjunto. A operação mais cara define a ordem de crescimento do módulo:

- Iniciar lista / professor: melhor caso O(1), caso médio O(1), pior caso O(1).
- Inserir: melhor caso O(1), caso médio O(1), pior caso O(1).
- Buscar: melhor caso O(1), caso médio O(N), pior caso O(N).
- Remover: melhor caso O(1), caso médio O(N), pior caso O(N).
- Listar: melhor caso O(N), caso médio O(N), pior caso O(N).
- Liberar: melhor caso O(N), caso médio O(N), pior caso O(N).

As operações de iniciar e inserir têm custo constante, enquanto buscar, remover, listar e liberar crescem linearmente com N. Logo, Professor.c implementado com lista duplamente encadeada **possui complexidade O(N) para suas operações mais custosas, e O(1) para iniciar e inserir.**

Comparação entre Aluno.c e Professor.c

As ordens de crescimento são as mesmas nas operações principais: inserir no fim é O(1), buscar é O(N) no pior caso e listar é O(N), tanto no vetor quanto na lista.

A diferença está na remoção: no vetor ela é O(N) em todos os casos, por causa do deslocamento de elementos, enquanto na lista ela tem melhor caso O(1) e pior caso O(N), pois só a busca depende de N.

Em contrapartida, **a lista não tem acesso direto por posição,** exige liberar a memória de cada nó no final e gasta mais memória por elemento (dois ponteiros por nó), enquanto o vetor tem capacidade máxima fixa.

Em resumo:

Na remoção, o vetor precisa copiar todos os elementos seguintes uma posição para trás, e esse deslocamento faz o custo ser **O(N) em qualquer caso**. Na lista basta religar os ponteiros dos vizinhos, um número fixo de passos, então só a busca depende de N e o melhor caso é O(1).

Na memória, o vetor guarda só os dados, enquanto cada nó da lista gasta dois ponteiros extras (`prox` e `ant`) mais o controle do `malloc`. Em contrapartida, o vetor reserva a capacidade máxima desde o início, e a lista usa só o que precisa.

O vetor compensa mais para Aluno e Professor. Ele usa menos memória, é mais rápido para buscar e listar já que permite acessar elementos por índice , e empata com a lista em inserir, que é o uso mais comum.

A lista só vence na remoção, por não precisar deslocar elementos, e quando o número de cadastros é imprevisível, já que o vetor tem um teto fixo.